

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
CARRERA DE CIENCIAS DE LA COMPUTACIÓN



**IMPLEMENTACIÓN DE UN PROCESADOR PIPELINE EN BASE A UN SINGLE
CYCLE**

AUTOR(ES)

Marco Wanly Obregón Casique

Ferrel Valentino Infante Garcia

Yuri Abel Escobar

ASESOR(ES)

Williams Valencia, Carlos Eduardo

Lima – Perú

2026

ÍNDICE

1. Introducción	8
2. Fundamentos Teóricos del Pipeline	8
2.1 Arquitectura Pipeline	9
Instruction Fetch (IF)	9
Instruction Decode (ID)	10
Execute (EX)	10
Memory Access (MEM)	10
Write Back (WB)	10
2.2 Datapath del Procesador	10
2.3 Unidad de Control	11
3. Implementación de las Instrucciones del ISA	12
3.1 Instrucciones Tipo R	13
3.2 Instrucciones Tipo I	14
3.3 Instrucciones de Memoria	14
Load Word (lw)	14
Store Word (sw)	14
3.4 Instrucciones de Control	14
3.5 Cambios Realizados en el Código	15
3.5.1 Segmentación del Datapath	15
3.5.2 Extensión del Set de Instrucciones	16
3.5.3 Implementación de Nuevas Instrucciones	17
3.5.4 Implementación Correcta de Branches	18
3.5.5 Implementación de JALR	19
3.5.6 Corrección del Banco de Registros	20
3.5.7 Modificaciones en la Memoria de Instrucciones	20

4. Programa de Prueba del ISA Sin Dependencias	21
4.1 Objetivo	21
4.2 Programa Utilizado	22
4.3 Archivo de Memoria	22
Programa utilizado: programa1_no_dependencias.mem	22
4.4 Resultados Esperados	23
4.5 Waveforms	23
4.6 Análisis de Resultados	24
5. Fundamentos Teóricos del Hazard Unit	24
5.1 Data Hazards	25
5.2 Load-Use Hazards	25
5.3 Control Hazards	26
5.4 Técnicas de Resolución	26
6. Implementación del Hazard Unit	27
6.1 Diseño General	27
6.2 Cambios Realizados en el Código	27
6.3 Datapath con Hazard Unit	27
7. Pruebas del Hazard Unit	28
7.1 Programa 2: Prueba de Forwarding	28
Objetivo	28
Código Utilizado	28
Ejecución Sin Hazard Unit	29
Ejecución Con Hazard Unit	29
Análisis	30
7.2 Programa 3: Prueba de Stalling	30
Objetivo	31

Código Utilizado	31
Ejecución Sin Hazard Unit	31
Ejecución Con Hazard Unit	32
Análisis	32
7.3 Programa 4: Prueba de Flushing	33
Objetivo	33
Código Utilizado	33
Ejecución Sin Hazard Unit	33
Ejecución Con Hazard Unit	34
Análisis	34
8. Conclusiones	35

1. Introducción

Los procesadores modernos utilizan técnicas de segmentación (pipeline) para incrementar el paralelismo a nivel de instrucciones y mejorar el rendimiento del sistema. En lugar de ejecutar una instrucción completamente antes de iniciar la siguiente, el pipeline divide la ejecución en varias etapas, permitiendo que múltiples instrucciones se procesen simultáneamente en distintas fases.

En este proyecto se implementó un procesador RISC-V pipelined de cinco etapas utilizando Verilog HDL. El diseño incluye soporte para las instrucciones especificadas en el ISA base proporcionado en el enunciado, así como una unidad de detección y resolución de hazards capaz de manejar dependencias de datos y control mediante técnicas de forwarding, stalling y flushing.

El objetivo principal de esta primera parte del proyecto es construir un procesador funcional que sirva como base para la futura implementación de la extensión atómica A de RISC-V. Para verificar la correctitud del diseño se desarrollaron diversos programas de prueba y testbenches que permiten observar el comportamiento interno del pipeline mediante simulaciones y waveforms.

2. Fundamentos Teóricos del Pipeline

2.1 Arquitectura Pipeline

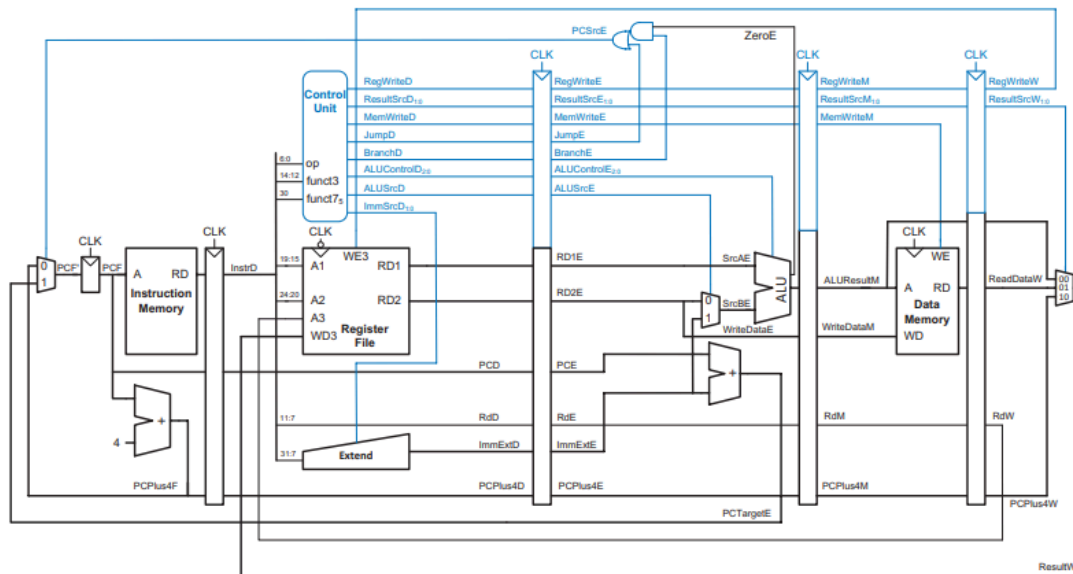


Figura 1. Arquitectura Pipeline

La técnica de pipeline consiste en dividir la ejecución de una instrucción en múltiples etapas independientes. De esta forma, diferentes instrucciones pueden ejecutarse simultáneamente en distintas etapas del procesador, incrementando el throughput del sistema.

El procesador implementado posee cinco etapas:

Instruction Fetch (IF)

Durante esta etapa se obtiene la instrucción desde la memoria de instrucciones utilizando el valor actual del Program Counter (PC). Además, se calcula la dirección de la siguiente instrucción.

Instruction Decode (ID)

La instrucción es decodificada y se leen los operandos necesarios desde el banco de registros. También se genera la información de control requerida para las etapas posteriores.

Execute (EX)

Se realizan las operaciones aritméticas y lógicas mediante la ALU. Asimismo, se calculan las direcciones efectivas para accesos a memoria y se evalúan las condiciones de salto y branch.

Memory Access (MEM)

Se ejecutan las operaciones de lectura y escritura sobre la memoria de datos.

Write Back (WB)

El resultado final de la instrucción es escrito en el banco de registros.

2.2 Datapath del Procesador

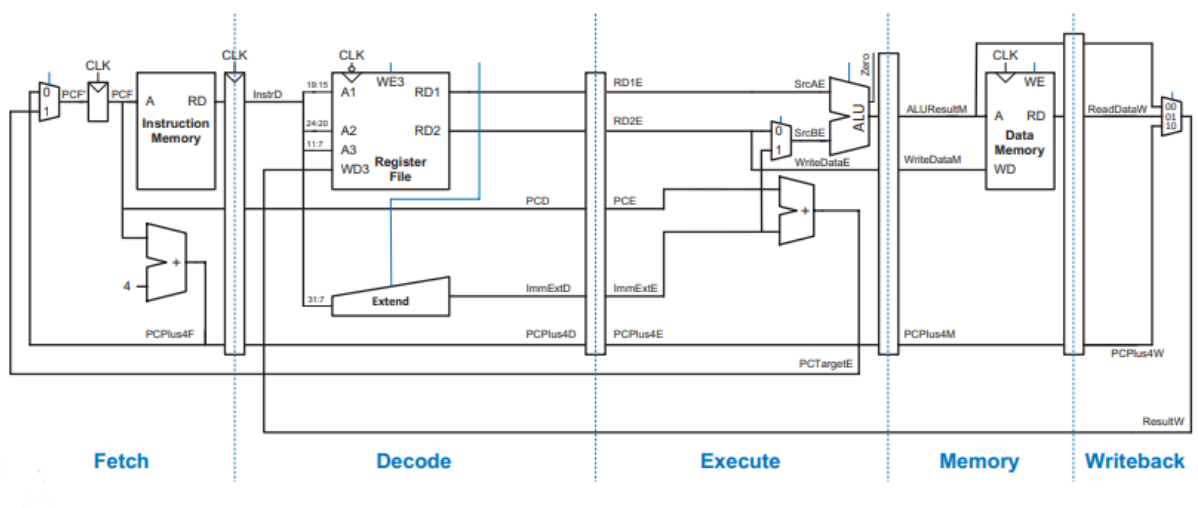


Figura 2. Datapath del Procesador

El datapath implementado está compuesto por los bloques funcionales principales del procesador, incluyendo el banco de registros, la unidad de control, la ALU, la memoria de instrucciones, la memoria de datos y los registros de pipeline que separan las distintas etapas.

Los registros de pipeline permiten almacenar temporalmente la información generada en cada etapa para ser utilizada por la siguiente durante el ciclo de reloj posterior.

2.3 Unidad de Control

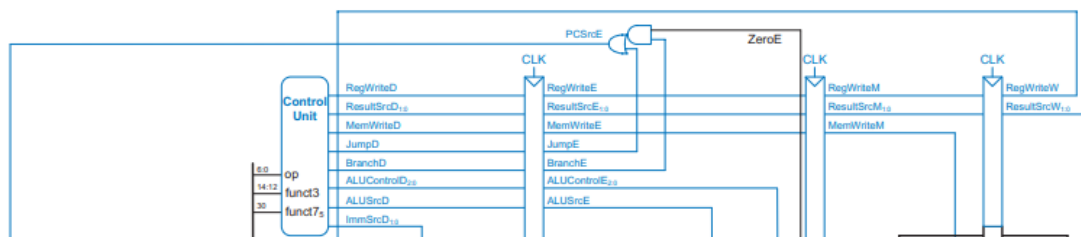


Figura 3. Diagrama de la Unidad de Control

La unidad de control es responsable de generar las señales necesarias para coordinar el funcionamiento de los distintos componentes del procesador.

Entre las principales señales generadas se encuentran:

- RegWrite, MemWrite, ResultSrc, ALUSrc, Branch, Jump, ImmSrc, ALUControl

Estas señales son propagadas a través de los registros de pipeline para asegurar que cada instrucción mantenga su contexto de ejecución.

3. Implementación de las Instrucciones del ISA

Se implementaron todas las instrucciones especificadas en el Cuadro 1 del enunciado del proyecto.

Instrucción RVC	Equivalente de 32 bits	Descripción
c.lw rd', imm(rs1')	lw rd, imm(rs1)	Load word
c.sw rs2', imm(rs1')	sw rs2, imm(rs1)	Store word
c.lwsp rd, imm(x2)	lw rd, imm(x2)	Load word from stack pointer
c.swsp rs2, imm(x2)	sw rs2, imm(x2)	Store word to stack pointer
c.addi rd, imm	addi rd, rd, imm	Add immediate
c.slli rd, shamt	slli rd, rd, shamt	Shift left logical immediate
c.srli rd', shamt	srli rd, rd, shamt	Shift right logical immediate
c.srai rd', shamt	srai rd, rd, shamt	Shift right arithmetic immediate
c.andi rd', imm	andi rd, rd, imm	AND immediate
c.add rd, rs2	add rd, rd, rs2	Add
c.sub rd', rs2'	sub rd, rd, rs2	Subtract
c.xor rd', rs2'	xor rd, rd, rs2	XOR
c.or rd', rs2'	or rd, rd, rs2	OR
c.and rd', rs2'	and rd, rd, rs2	AND
c.lui rd, nzimm	lui rd, nzimm	Load upper immediate
c.beqz rs1', offset	beq rs1, x0, offset	Branch if zero
c.bnez rs1', offset	bne rs1, x0, offset	Branch si not zero
c.j offset	jal x0, offset	Jump
c.jal offset	jal x1, offset	Jump and link (RV32C only)
c.jr rs1	jalr x0, rs1, 0	Jump register
c.jalr rs1	jalr x1, rs1, 0	Jump and link register

Cuadro 1: Instrucciones RVC a implementar. La notación rd', rs1', rs2' indica registros restringidos al rango x8–x15.

Figura 4. Tabla de Instrucciones ISA

3.1 Instrucciones Tipo R

Las instrucciones tipo R utilizan dos registros fuente y un registro destino.

Las instrucciones implementadas fueron:

- add, sub, and, or, xor, sll, srl, sra

Estas instrucciones son ejecutadas completamente por la ALU durante la etapa EX.

3.2 Instrucciones Tipo I

Las instrucciones tipo I utilizan un operando inmediato generado por la unidad de extensión de inmediatos.

Las instrucciones implementadas fueron:

- addi, andi, ori, xori, slli, srli, srai

Para estas instrucciones fue necesario implementar la lógica de extensión y selección de inmediatos correspondiente.

3.3 Instrucciones de Memoria

Load Word (lw)

La instrucción lw calcula una dirección efectiva utilizando un registro base y un desplazamiento inmediato. Posteriormente realiza una lectura desde memoria y escribe el resultado en el registro destino.

Store Word (sw)

La instrucción sw calcula una dirección efectiva y almacena el contenido de un registro en la memoria de datos.

3.4 Instrucciones de Control

Las instrucciones de control implementadas fueron:

- beq, bne, blt, bge, jal, jalr

Estas instrucciones modifican el flujo normal de ejecución del programa actualizando el Program Counter con una nueva dirección.

3.5 Cambios Realizados en el Código

A partir del procesador single-cycle desarrollado previamente, fue necesario realizar diversas modificaciones para soportar la ejecución segmentada en cinco etapas, ampliar el conjunto de instrucciones implementadas y añadir mecanismos para resolver hazards.

3.5.1 Segmentación del Datapath

El procesador original de un solo ciclo fue reorganizado en las etapas IF, ID, EX, MEM y WB. Para ello se añadieron registros de pipeline entre cada etapa, permitiendo almacenar tanto señales de datos como señales de control necesarias para la siguiente etapa.

Se implementaron los registros:

- IF/ID
- ID/EX
- EX/MEM
- MEM/WB

Esto permitió ejecutar varias instrucciones simultáneamente en distintas etapas del pipeline.

3.5.2 Extensión del Set de Instrucciones

El diseño inicial solamente soportaba las instrucciones básicas:

- lw, sw, add, sub, and, or, addi, andi, ori, slti, beq, jal

Para cumplir con los requerimientos del proyecto fue necesario extender la unidad de control y la ALU para soportar las 24 instrucciones especificadas en el Cuadro 1.

Modificaciones en la ALU

La señal ALUControl fue ampliada de 3 bits a 4 bits para incorporar las operaciones:

- ADD, SUB, AND, OR, XOR, SLT, SLL, SRL, SRA, PASSB (utilizada para LUI)

Modificaciones en la generación de inmediatos

La señal ImmSrc fue ampliada de 2 bits a 3 bits para soportar los distintos formatos del ISA:

Tabla 1. Códigos de Inmediatos

Tipo	Código
I	000
S	001
B	010
J	011
U	100

Gracias a ello fue posible implementar la instrucción LUI.

3.5.3 Implementación de Nuevas Instrucciones

Se añadieron las siguientes instrucciones:

Instrucciones tipo I

- xori, slli, srli, srai

Instrucciones tipo R

- xor, sll, srl, sra

Instrucciones de control

- bne, blt, bge, jalr

Instrucción U

- lui

Para ello se modificaron los módulos:

- maindec.v, aludec.v, alu.v, extend.v

3.5.4 Implementación Correcta de Branches

Inicialmente todos los branches eran tratados como instrucciones beq.

Posteriormente se añadió lógica de comparación en la etapa EX utilizando dos señales:

- eq : igualdad entre operandos.
- lt : comparación signed menor que.

En función del campo funct3 se selecciona la condición correspondiente:

- Para “beq”, se selecciona eq
- Para bne se selecciona $\sim$ eq
- Para blt se selecciona lt
- Para bge se selecciona $\sim$ lt

De esta manera se consiguió implementar correctamente todas las instrucciones de branch requeridas.

3.5.5 Implementación de JALR

El diseño inicial únicamente soportaba saltos relativos al PC.

Se incorporó una nueva señal JalrE y un nuevo camino de datos para calcular:

$$PCJalrE = rs1 + imm$$

cumpliendo con la especificación de RISC-V.

La señal PCSrcE fue ampliada a dos bits:

Tabla 2. Señales PCSrcE

PCSrcE	Selección
00	PC + 4
01	Branch tomado o JAL
10	JALR

Además, el multiplexor encargado de seleccionar el siguiente PC fue modificado de un mux de dos entradas a un mux de tres entradas.

3.5.6 Corrección del Banco de Registros

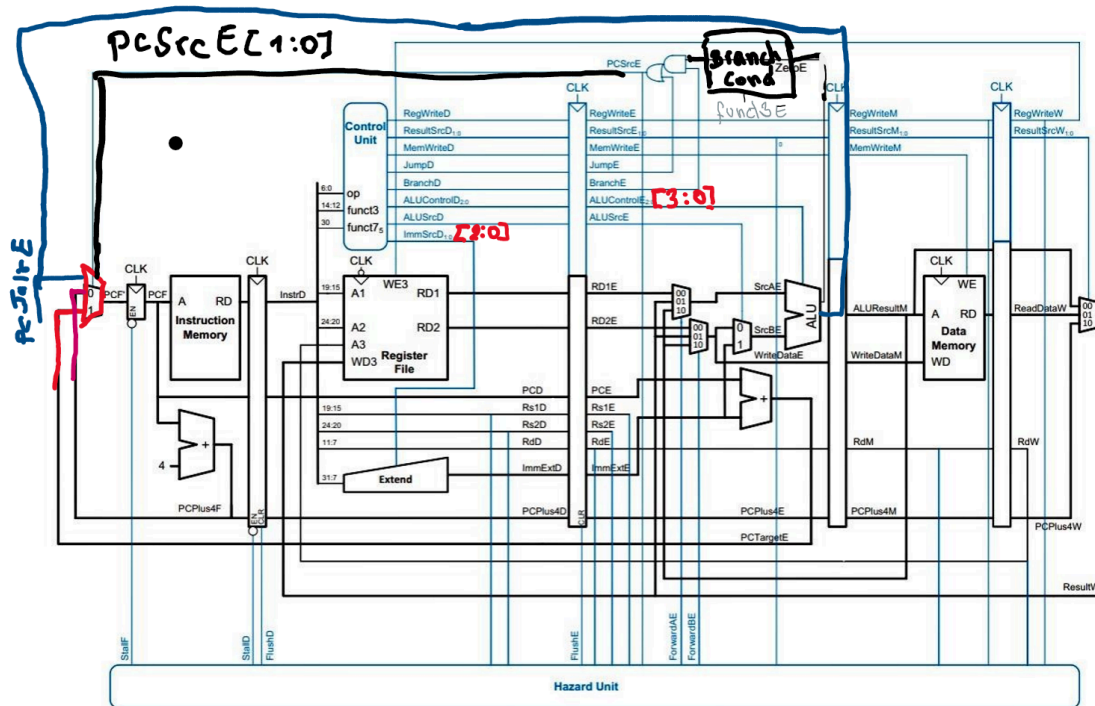
Durante las simulaciones se observó una condición de carrera entre la etapa WB y la lectura realizada en ID. Para evitar este problema, las escrituras del register file fueron trasladadas al flanco negativo del reloj, garantizando que el valor escrito por WB esté disponible antes de la lectura realizada por ID dentro del mismo periodo.

3.5.7 Modificaciones en la Memoria de Instrucciones

El nombre del archivo .mem era propagado a través de varios niveles jerárquicos utilizando vectores de bits, lo que producía errores durante la carga mediante \$readmemh.

La implementación fue modificada para utilizar parámetros tipo string, permitiendo una carga confiable de los programas de prueba.

3.6 Datapath Final



El datapath final incluye todas las rutas de datos necesarias para ejecutar correctamente las instrucciones del ISA requerido.

4. Programa de Prueba del ISA Sin Dependencias

4.1 Objetivo

El propósito de esta prueba es verificar el funcionamiento correcto de todas las instrucciones implementadas en un escenario sin dependencias de datos entre instrucciones consecutivas.

4.2 Programa Utilizado

Tabla 3. Código de Prueba 1

```
00500093 // addi x1, x0, 5
00a00113 // addi x2, x0, 10
00f00193 // addi x3, x0, 15
01400213 // addi x4, x0, 20
00100593 // addi x11, x0, 1
00200613 // addi x12, x0, 2
002082b3 // add x5, x1, x2
40320333 // sub x6, x4, x3
0040c4b3 // xor x9, x1, x4
06400513 // addi x10, x0, 100
00000063 // beq x0, x0, 0
```

4.3 Archivo de Memoria

Programa utilizado: programa1_no_dependencias.mem

4.4 Resultados Esperados

Tabla 4. Resultados de la Prueba 1

Registr o	Decima l	Hexadecim al
x1	5	0x00000005
x2	10	0x0000000A
x3	15	0x0000000F
x4	20	0x00000014
x5	15	0x0000000F
x6	5	0x00000005
x9	17	0x00000011
x10	100	0x00000064
x11	1	0x00000001
x12	2	0x00000002

4.5 Waveforms

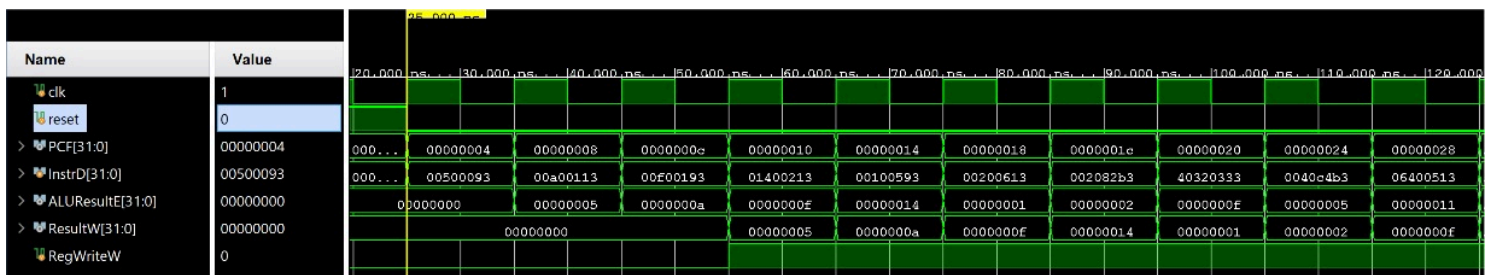


Figura 5. Waveform de Prueba 1

Las señales observadas durante la simulación incluyen:

- PC, InstrD, ALUResultE, ReadDataM, ResultW, RegWriteW

4.6 Análisis de Resultados

Los resultados obtenidos coinciden con los valores esperados para cada una de las instrucciones ejecutadas. Esto confirma el correcto funcionamiento del datapath, la unidad de control y los módulos auxiliares implementados. Los valores observados en las señales de Write Back y en el banco de registros coinciden con los resultados esperados y las instrucciones aritméticas y lógicas fueron ejecutadas correctamente.

5. Fundamentos Teóricos del Hazard Unit

En un procesador pipelined pueden aparecer conflictos conocidos como hazards, los cuales impiden la ejecución correcta de las instrucciones si no son tratados adecuadamente.

5.1 Data Hazards

Los data hazards ocurren cuando una instrucción depende del resultado generado por una instrucción anterior que aún no ha finalizado su ejecución.

Un ejemplo es:

```
add x1, x2, x3
```

```
sub x4, x1, x5
```

La instrucción sub necesita el valor de x1 antes de que este haya sido escrito en el banco de registros.

5.2 Load-Use Hazards

Este tipo particular de hazard ocurre cuando una instrucción intenta utilizar inmediatamente un dato cargado desde memoria.

Ejemplo:

```
lw x1, 0(x2)
add x3, x1, x4
```

Debido a que la lectura de memoria se completa en una etapa posterior, el valor requerido aún no está disponible.

5.3 Control Hazards

Los control hazards aparecen en instrucciones de salto o branch.

Ejemplo:

```
beq x1, x2, etiqueta_salto
```

Mientras se determina si el branch será tomado o no, el procesador puede haber comenzado a ejecutar instrucciones incorrectas.

5.4 Técnicas de Resolución

Para resolver estos conflictos se implementaron tres mecanismos:

- Forwarding
- Stalling
- Flushing

6. Implementación del Hazard Unit

6.1 Diseño General

La Hazard Unit implementada genera las siguientes señales de control:

- ForwardAE, ForwardBE, StallF, StallD, FlushD, FlushE

Estas señales permiten resolver automáticamente conflictos de datos y control durante la ejecución del pipeline.

6.2 Cambios Realizados en el Código

La incorporación del Hazard Unit requirió añadir un módulo de detección de conflictos y modificar las etapas del pipeline para que respondiera a sus señales de control.

6.2.1 Creación del módulo hazard_unit

Se añadió el módulo hazard_unit.v, el cual se encarga de detectar las dependencias de datos y control, también de generar las señales que las resuelven: ForwardAE y ForwardBE, StallF y StallD y FlushD y FlushE . Sus

puertos de entrada capturan los números de registro relevantes de cada etapa del pipeline:

- Rs1D, Rs2D (5 bits): registros fuente de la instrucción en ID.
- Rs1E, Rs2E (5 bits): registros fuente de la instrucción en EX.
- RdE, RdM, RdW (5 bits): registros destino de las instrucciones en EX, MEM y WB respectivamente.
- RegWriteM, RegWriteW (1 bit): indicadores de escritura en el banco de registros para las instrucciones en MEM y WB.
- MemReadE (1 bit): indica que la instrucción en EX es una carga de memoria (load).
- PCSrcTakenE (1 bit): indica que la instrucción en EX ha tomado un salto (branch, JAL o JALR).

6.2.2 Lógica de Forwarding

Para resolver las dependencias RAW se compara el registro fuente de la instrucción en EX contra los registros destino de las instrucciones en MEM y WB. Cuando coinciden y la instrucción correspondiente escribe en el banco de registros, se redirige el valor más reciente hacia la ALU. Se da prioridad a la etapa MEM código 2'b10 sobre la etapa WB código 2'b01, ya que MEM contiene el resultado más actual; si no hay coincidencia se usa el valor leído del banco de registros 2'b00. Se descarta cualquier reenvío cuando el registro fuente es x0, dado que siempre vale cero.

6.2.3 Multiplexores de Forwarding en la etapa EX

En ex_stage.v se agregaron dos multiplexores de tres entradas mux3 controlados por ForwardAE y ForwardBE. Estos seleccionan, para cada operando de la ALU, entre el valor original leído del banco de registros, el resultado proveniente de MEM o el resultado de WB, permitiendo inyectar el dato correcto sin esperar a que se complete la escritura en el banco de registros.

6.2.4 Detección de Load-Use y Stalling

Se añadió la detección del hazard load-use: cuando la instrucción en EX es una carga MemReadE y la instrucción en ID utiliza su registro destino como fuente, se activa la condición de stall. En ese caso se asertan StallF y StallD para congelar las etapas IF e IF/ID durante un ciclo. En `if_stage.v` se modificó el cálculo del próximo PC para que, ante StallF, el contador de programa mantenga su valor actual en lugar de avanzar. En `pipeline.v`, el registro IF/ID se mantiene sin actualizarse mientras StallD esté activo.

6.2.5 Inserción de Burbuja

Para que el ciclo de stall tenga efecto real, además de congelar IF e IF/ID se activa FlushE, lo que limpia el registro ID/EX e inserta una burbuja NOP en la etapa EX. De esta forma la instrucción dependiente espera el ciclo necesario para que el dato cargado desde memoria esté disponible antes de avanzar.

6.2.6 Flushing por Hazards de Control

Los hazards de control ocurren cuando una instrucción de branch, JAL o JALR modifica el flujo del programa. Dado que la dirección de salto se calcula y evalúa en la etapa EX, las instrucciones que ya ingresaron a las etapas IF e ID son especulativas y deben descartarse si el salto es tomado.

La señal PCSrcTakenE se activa cuando PCSrcE es distinto de 00 (que corresponde a PC+4), es decir, cuando la instrucción en EX genera un salto (branch tomado, JAL o JALR). Esta señal activa FlushD y FlushE de la siguiente manera:

```
assign FlushD = PCSrcTakenE;
```

```
assign FlushE = lwStall | PCSrcTakenE;
```

Donde FlushD limpia el registro IF/ID, descartando la instrucción que fue cargada especulativamente después del salto. FlushE limpia el registro ID/EX, descartando la instrucción que estaba en decodificación y que corresponde a la ruta especulativa.

6.2.7 Integración en el pipeline

En pipeline.v se instancia la Hazard Unit y se conectan todas sus señales a los puntos correspondientes del datapath:

```
hazard_unit hu (  
    .Rs1D(Rs1D), .Rs2D(Rs2D), .Rs1E(Rs1E), .Rs2E(Rs2E),  
    .RdE(RdE), .RdM(RdM), .RdW(RdW),  
    .RegWriteM(RegWriteM), .RegWriteW(RegWriteW),  
    .MemReadE(MemReadE), .PCSrcTakenE(PCSrcTakenE),  
    .StallF(StallF), .StallD(StallD), .FlushD(FlushD),  
    .FlushE(FlushE),  
    .ForwardAE(ForwardAE), .ForwardBE(ForwardBE)  
);
```

Cabe aclarar que las señales MemReadE y PCSrcTakenE se derivan localmente en pipeline.v a partir de señales de la etapa EX:

```
wire MemReadE = ResultSrcE[0];          // 1 si la instrucción en EX  
es lw (ResultSrc = 01)  
wire PCSrcTakenE = (PCSrcE != 2'b00); // 1 si el PC no sigue a  
PC+4
```

Por su parte el flujo de integración es el siguiente:

Forwarding: ForwardAE y ForwardBE se conectan a los multiplexores fwdA_mux y fwdB_mux en ex_stage.v, que seleccionan entre RD1E/RD2E, ResultW (desde WB) y ALUResultM (desde MEM).

Stalling: StallF se conecta a if_stage.v para congelar el PC; StallID controla la habilitación del registro IF/ID en pipeline.v.

Flushing: FlushD limpia el registro IF/ID y FlushE limpia el registro ID/EX, ambos en pipeline.v.

6.3 Datapath con Hazard Unit

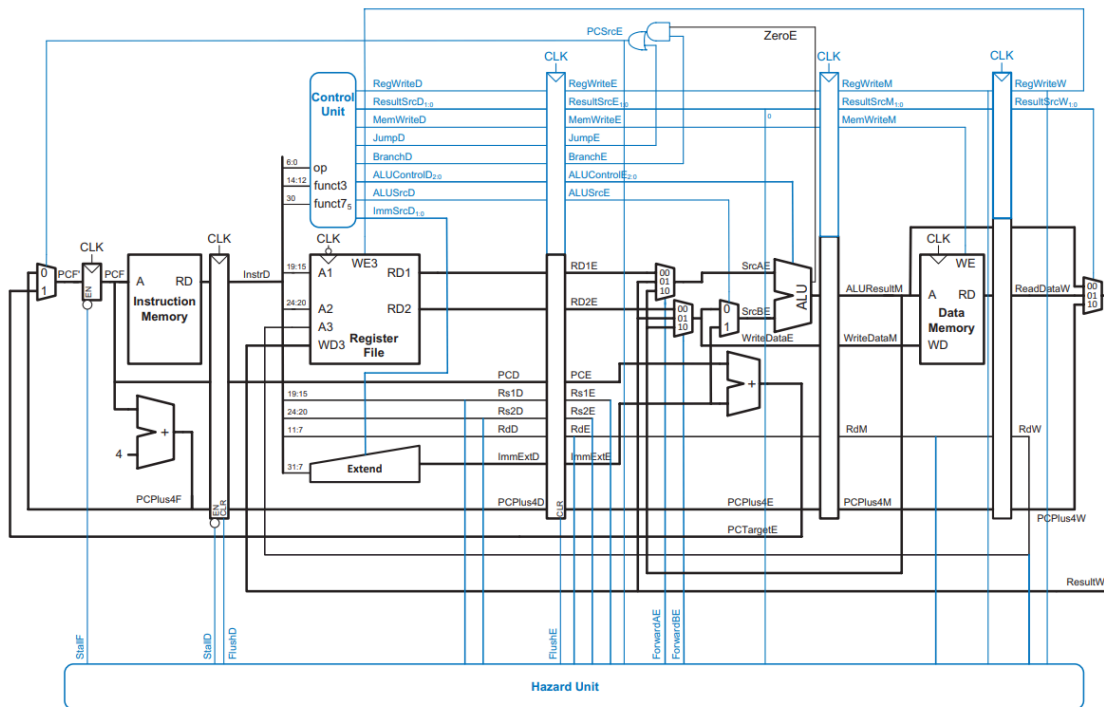


Figura 6. Datapath con Hazard Unit

7. Pruebas del Hazard Unit

7.1 Programa 2: Prueba de Forwarding

Objetivo

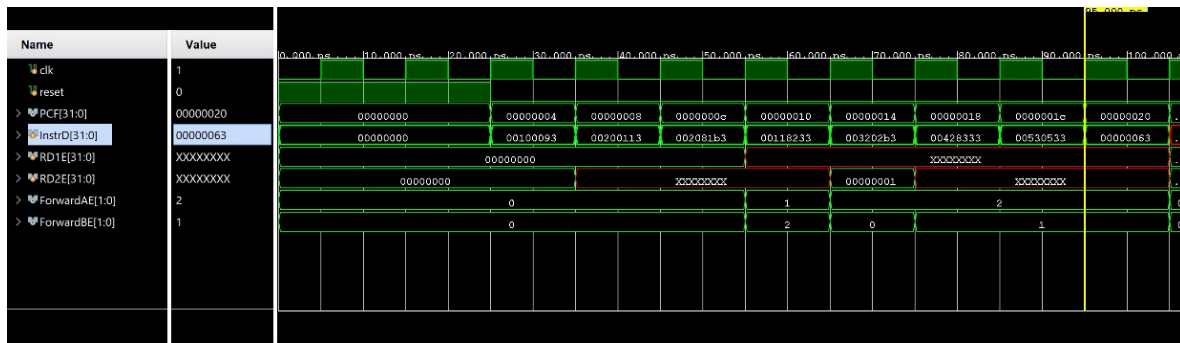
Verificar la correcta resolución de dependencias RAW mediante forwarding.

Código Utilizado

Tabla 5. Código de Prueba 2

```
00100093 // addi x1, x0, 1
00200113 // addi x2, x0, 2
002081b3 // add  x3, x1, x2
00118233 // add  x4, x3, x1
003202b3 // add  x5, x4, x3
00428333 // add  x6, x5, x4
00530533 // add  x10, x6, x5
00000063 // beq  x0, x0, 0
```

Ejecución Sin Hazard Unit



Se observa que la instrucción dependiente utiliza un valor incorrecto debido a que el resultado aún no ha sido escrito en el banco de registros.

Figura 7. Ejecución Sin Hazard Unit (Forwarding)

Ejecución Con Hazard Unit

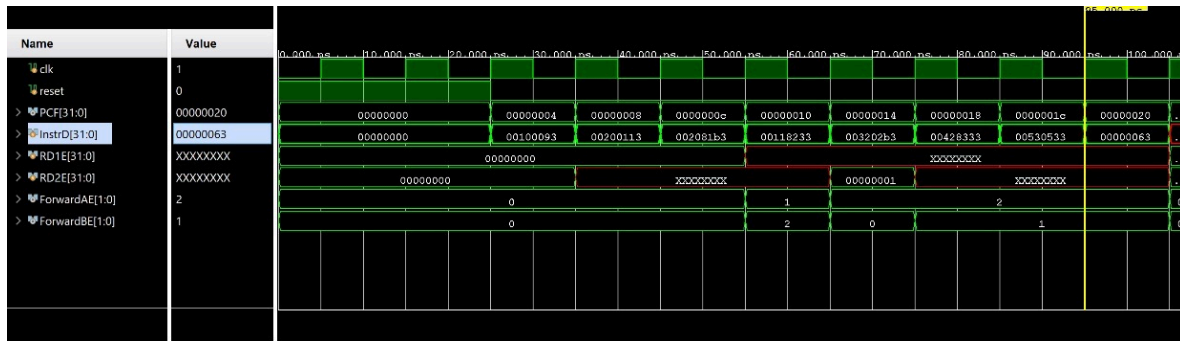


Figura 8. Ejecución Con Hazard Unit (Forwarding)

La señal ForwardAE/ForwardBE selecciona correctamente el resultado proveniente de etapas posteriores del pipeline.

Análisis

Al ejecutar el programa sin la Hazard Unit, las instrucciones con dependencias RAW leen valores que aún no han sido escritos en el banco de registros, produciendo resultados incorrectos y señales indefinidas XXXXXXXXX en la salida de la ALU. En cambio, al habilitar la Hazard Unit se observa la activación de las

señales ForwardAE y ForwardBE, las cuales permiten reenviar los resultados desde las etapas EX/MEM y MEM/WB hacia la etapa EX, evitando esperas y garantizando que cada instrucción reciba los operandos correctos. Como resultado, la cadena de dependencias se resuelve correctamente y se obtienen los valores esperados en los registros finales.

7.2 Programa 3: Prueba de Stalling

Objetivo

Verificar la correcta detección de hazards tipo load-use.

Código Utilizado

Tabla 6. Código de Prueba 3

```
00400093 // addi x1, x0, 4
00102023 // sw  x1, 0(x0)
00002103 // lw  x2, 0(x0)
00210313 // addi x6, x2, 2
00900193 // addi x3, x0, 9
00318393 // addi x7, x3, 3
00000063 // beq  x0, x0, 0
```

Ejecución Sin Hazard Unit

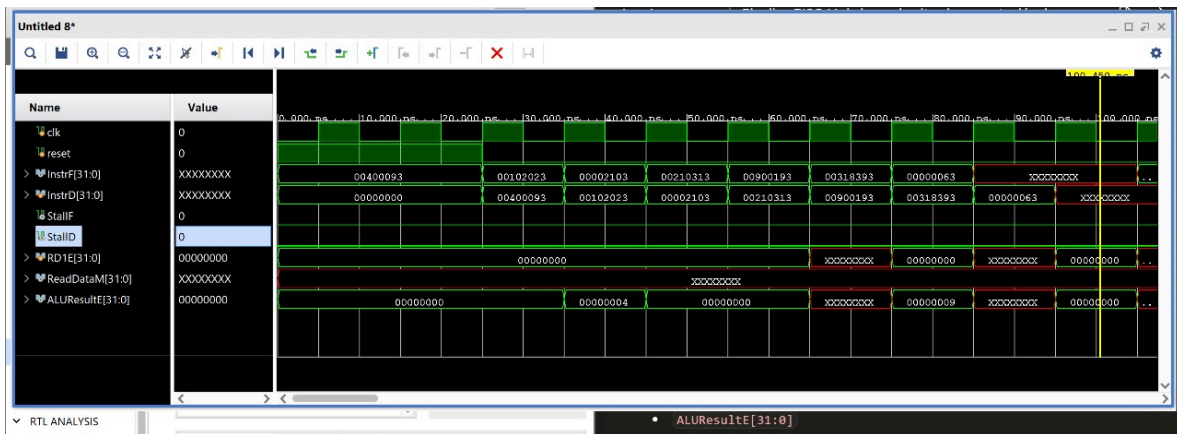
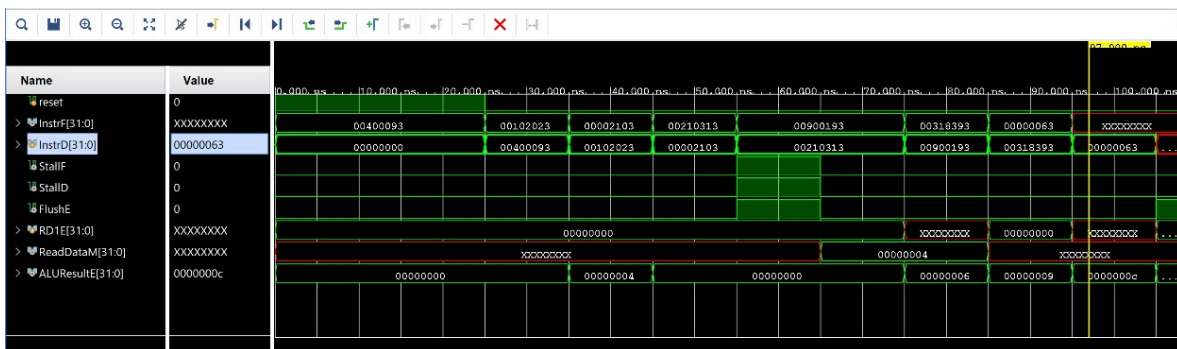


Figura 9. Ejecución Sin Hazard Unit (Stalling)

Se observa una lectura incorrecta debido a que el dato aún no se encuentra disponible.

Ejecución Con Hazard Unit



Las señales StallF y StallID detienen temporalmente el pipeline mientras FlushE inserta una burbuja.

Figura 10. Ejecución Con Hazard Unit (Stalling)

Análisis

El programa genera una dependencia de tipo load-use, ya que la instrucción `addi x6, x2, 2` utiliza inmediatamente el valor cargado por `lw x2, 0(x0)`. En la simulación sin Hazard Unit, la instrucción dependiente se ejecuta antes de que el dato leído de memoria esté disponible, produciendo valores indefinidos (XXXXXXXX) y resultados incorrectos. En cambio, con la Hazard Unit habilitada se observa la activación de las señales `StallF`, `StallD` y `FlushE`, las cuales detienen temporalmente el pipeline e insertan una burbuja en la etapa EX. Esto permite que el valor leído por `lw` llegue correctamente desde memoria antes de ser utilizado, obteniéndose los resultados esperados $x2 = 4$, $x6 = 6$, $x3 = 9$ y $x7 = 12$, verificando el correcto funcionamiento del mecanismo de stalling para resolver hazards de tipo load-use.

7.3 Programa 4: Prueba de Flushing

Objetivo

Verificar la correcta eliminación de instrucciones inválidas luego de un branch o jump.

Código Utilizado

Tabla 7. Código de Prueba 4

```
00100093 // addi x1, x0, 1
00200113 // addi x2, x0, 2
00208863 // beq  x1, x2, 16
00308193 // addi x3, x1, 3
00410213 // addi x4, x2, 4
00108463 // beq  x1, x1, 8
3e700f13 // addi x30, x0, 999
00700293 // addi x5, x0, 7
0080006f // jal  x0, 8
37800f93 // addi x31, x0, 888
00900313 // addi x6, x0, 9
00000063 // beq  x0, x0, 0
```

Ejecución Sin Hazard Unit

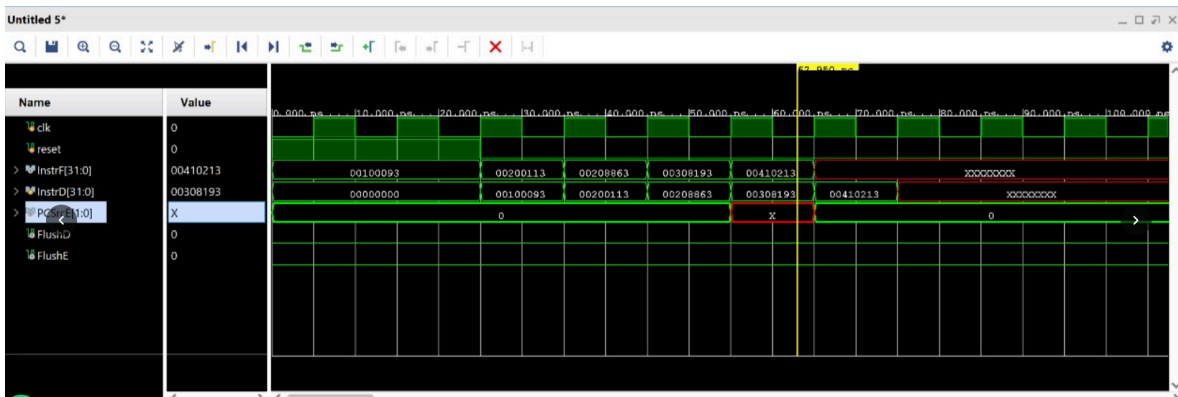


Figura 11. Ejecución Sin Hazard Unit (Flushing)

Se observa que instrucciones incorrectas continúan ejecutándose después de un salto.

Ejecución Con Hazard Unit

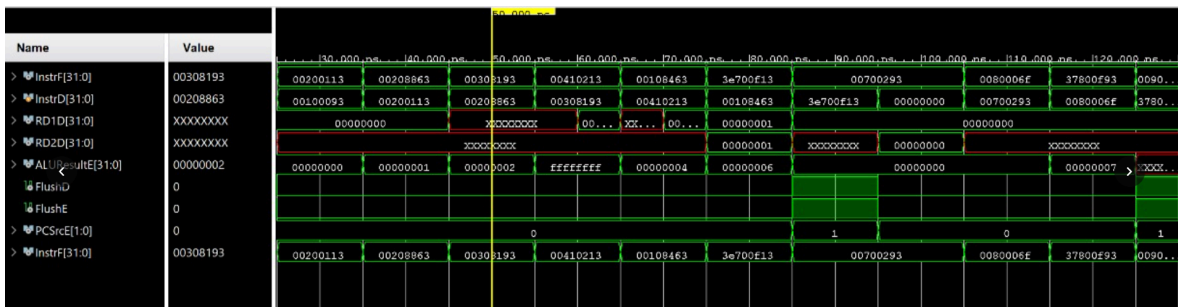


Figura 12. Ejecución Con Hazard Unit (Flushing)

Las señales FlushD y FlushE eliminan las instrucciones especulativas presentes en el pipeline.

Análisis

El programa genera hazards de control mediante instrucciones beq y jal, las cuales modifican el flujo normal de ejecución. En la simulación sin Hazard Unit, las

instrucciones que ya habían sido cargadas en las etapas IF e ID continúan ejecutándose aún cuando el salto ha sido tomado, provocando la ejecución incorrecta de instrucciones que debían descartarse. Con la Hazard Unit habilitada, se observa la activación de las señales PCSrcE, FlushD y FlushE, las cuales eliminan las instrucciones especulativas presentes en el pipeline e introducen burbujas cuando se produce un branch tomado o un jump. De esta forma, instrucciones como `addi x30, x0, 999` y `addi x31, x0, 888`, que pertenecen al camino incorrecto, son descartadas y solo se ejecutan las instrucciones válidas, obteniéndose el flujo de programa esperado y verificando el correcto funcionamiento del mecanismo de flushing para resolver hazards de control.

8. Conclusiones

En este proyecto se implementó satisfactoriamente un procesador RISC-V pipelined de cinco etapas utilizando Verilog HDL. Se verificó el correcto funcionamiento de todas las instrucciones requeridas mediante programas de prueba específicos y simulaciones funcionales.

Asimismo, se diseñó e integró una Hazard Unit capaz de resolver dependencias de datos mediante forwarding, detectar conflictos load-use mediante stalling y corregir hazards de control mediante flushing. Los resultados obtenidos en las simulaciones demuestran el correcto funcionamiento del pipeline bajo distintos escenarios de ejecución.

La implementación desarrollada constituye una base sólida para la siguiente etapa del proyecto, en la cual se incorporará la extensión atómica A del ISA RISC-V.